

AegisAI

A reproducible framework for testing whether AI systems can distinguish trusted instructions from untrusted content in text and images.

Project repository <https://github.com/zarbid62/text-image-attack-defence-framework>

The Problem

AI applications increasingly process documents, screenshots, images, OCR text, web content, and tool responses. Some of that content may contain instructions designed to redirect the model.

This is a prompt-injection risk. The content looks like data, but it attempts to behave like a higher-priority instruction.

What AegisAI asks

- Can the system identify instruction-like content?
- Can it preserve the trusted task boundary?
- Can it prevent untrusted text from triggering tools?
- Can different models and defenses be compared fairly?

How It Works

AegisAI follows a simple security workflow: keep the trusted task separate, test the untrusted content, compare defenses, and preserve the evidence.

1. Load a versioned attack or benign sample.
1. Keep the trusted review task separate from untrusted content.
1. Run an undefended baseline and a defended pipeline.
1. Evaluate local or hosted LLM and vision models.
1. Record detection, bypasses, accuracy, errors, confidence, and latency.
1. Generate understandable reports for technical and non-technical review.

Technology and Engineering

CPU-first accessibility

The project includes `8gb`, `16gb`, and larger CPU profiles. This allows students, reviewers, and researchers to reproduce a controlled image test on ordinary hardware without requiring a GPU.

CPU image testing guidehttps://github.com/zarbid62/text-image-attack-defence-framework/blob/main/aegisai_text_injection_test/image_injection_test/cpu_based_image_test/README.md

Evidence and Value

What is measured

Attack detectionAttack bypassAccuracyFalse positivesConfidenceErrorsLatency

Why this is more than a demo

- Attack and benign samples are versioned.
- The same conditions can be compared across models.
- Each run records its command, configuration, prompts, and timestamp.
- Previous results are archived instead of silently overwritten.
- Known bypasses and limitations are reported honestly.

Career relevance

This project demonstrates AI engineering, LLM evaluation, AI-agent safety, multimodal systems, prompt engineering, cybersecurity, Python, Node.js, data analysis, experiment design, and responsible AI.

Why I Built It

I built AegisAI to study a practical AI security question: how can an AI system use external information without allowing that information to silently change its instructions?

The project turns that question into an end-to-end experiment. It combines attack design, trust-boundary enforcement, model evaluation, CPU-based testing, structured metrics, and clear reporting in one reproducible workflow.

What I learned

Reliable AI security requires layered controls. A detector may miss subtle language, so the system must also preserve the trusted task boundary and prevent untrusted content from directly becoming a tool command.

GitHub<https://github.com/zarbid62/text-image-attack-defence-framework>